



Informe Laboratorio 6 — Avance Proyecto Unidad II

Sistema de Gestión para Supermercado

Asignatura: Programación Avanzada

Integrantes: Santiago Farfán, Cristián Ruiz y Sebastián Ruiz.

Fecha: 8 de octubre de 2026.

1. **Introducción:** El presente informe documenta el avance del proyecto correspondiente a la Unidad II de Programación Avanzada. El dominio elegido es un Sistema de Gestión para un Supermercado, cuyo objetivo es permitir administrar productos, compradores y ventas mediante un menú interactivo por consola. Este avance incluye la estructura base del sistema: jerarquía de productos, interfaces, enumeraciones, colecciones y persistencia mediante serialización de objetos.

2. Supuestos de diseño y decisiones del equipo

2.1 Organización por paquetes: Se organizó el proyecto en los paquetes modelo, sistema, interfaces y enums. Esta separación permite mantener el código ordenado y facilitar el mantenimiento.

2.2 Jerarquía de herencia: Se decidió crear la clase abstracta Producto, porque todos los artículos del supermercado comparten atributos comunes (identificador, denominación, valor base, stock y unidad de medida) y un comportamiento común (calcular un precio). El método calcularPrecio() se definió como abstracto porque cada tipo de producto aplica reglas de precio distintas. Las subclases Lacteo, Verdura y Limpieza representan categorías reales del dominio.

2.3 Uso de interfaces

Se implementaron dos interfaces:

- **Rankeable:** La implementan Producto y Comprador, lo que permite obtener un criterio numérico de ranking.

- Registrable: La implementa GuardadoSeguro, lo que define el comportamiento de guardado y la carga de datos.

Se decidió separar la persistencia de una clase aparte (GuardadoSeguro) para que ControladorSupermercado no se encargue directamente de leer y escribir archivos.

2.4 Colecciones: Se utiliza HashMap<String, Producto> en la clase Inventario porque la búsqueda por identificador es la operación más frecuente. Se usa ArrayList para listados como boletas y compradores.

2.5 Persistencia: Se eligió serialización de objetos porque es sencilla de implementar en Java y permite guardar el estado del sistema en un archivo binario (data/supermercado.dat).

2.6 Decisiones propias del avance: Para este avance se prefirió usar ciclos while en lugar de for, y se simplificó la presentación de objetos eliminando los métodos toString() personalizados. También se redujo la cantidad de compradores precargados a dos, dejando seis productos (dos de cada tipo).

3. Descripción de clases

clase	descripción
Producto	Clase abstracta base para todos los productos. Nos define atributos comunes y el método abstracto que tenemos como "calcularPrecio(). Implementa Rankeable y Serializable.
lácteo	Producto lácteo. Aplica un 15% de recargo si requiere refrigeración.
verdura	Producto vegetal. Aplica 20% de recargo si es orgánica.
limpieza	Producto de limpieza. Aplica un 10% de impuesto si es tóxico y un 5% de descuento si no lo es.
comprador	Representa el cliente del super. Mantiene un historial de boletas. Implementa tambien un Rankeable.

boleto	Registra una compra. Contiene un comprador, una lista de productos, el total a pagar y una categoría de oferta.
inventario	Gestiona los productos mediante un hashmap indexado por identificador.

Paquete enums

Enum	Descripción
Medida	Unidades de medida como: kilogramo, litro y unidad.
CategoriaOferta	Representa tipos de oferta: DOSPORUNO, REBAJA_PORCENTUAL

Paquete de interfaces

interfaz	Descripción
Rankeable	Define el método de obtenerCriterioRanking()
registrable	Define los métodos guardarDatos() y cargarDatos()

4. **Justificación del uso de clases abstractas:** Se utilizó una clase abstracta (Producto) porque representa una generalización del dominio que no debería instanciarse directamente. Todos los productos comparten estado y comportamiento, pero cada subtipo define su propia lógica de precio. El uso de abstract garantiza que cada subclase implemente calcularPrecio(), aprovechando el polimorfismo.
5. **Función de las interfaces:** Rankeable Permite que tanto productos como compradores puedan ser evaluados mediante un criterio numérico común. Esto facilita la futura implementación de rankings. Registrable Establece un contrato para clases que deben persistir su estado. ControladorSupermercado la implementa, encapsulando la lógica de guardado y carga.
6. **Estrategia de colecciones:**

- HashMap<String, Producto> en Inventario: justificado porque la búsqueda por ID es la operación principal.
 - ArrayList en Comprador y ControladorSupermercado: mantiene el historial de transacciones.
 - ArrayList en ControladorSupermercado: permite recorrer los clientes registrados.
7. **Persistencia:** La persistencia se implementó mediante serialización de objetos. El controlador guarda tres objetos en el archivo data/supermercado.dat:
- El inventario.
 - La lista de compradores.
 - El registro de boletas. Al iniciar el sistema se intenta cargar el archivo. Si no existe, se precargan datos de prueba.
8. **Pruebas realizadas, se verificaron los siguientes escenarios:**
- Compilación exitosa del proyecto.
 - Carga de datos precargados al iniciar sin archivo previo.
 - Listado de productos y compradores desde el menú.
 - Venta exitosa con cálculo polimórfico del precio y descuento.
 - Persistencia: cierre y reapertura del programa conservando datos.
 - Manejo de errores básico: comprador no encontrado, producto inexistente, stock insuficiente.
9. **Conclusión:**
- En este avance se logró implementar la estructura orientada a objetos del sistema de supermercado, cumpliendo con los requisitos del laboratorio: paquetes, herencia con clase abstracta, interfaces, enums, colecciones, menú con scanner y persistencia serializable. El sistema compila y ejecuta sin errores, y las funcionalidades básicas de venta y guardado que fueron probadas exitosamente. Para la entrega final se espera completar funcionalidades adicionales como búsquedas avanzadas, rankings completos, manejo más robusto de excepciones y una interfaz de usuario más amigable.